AI ASSISTED CODING
LAB-ASSIGNMENT-18.1

A.Srichala
2303A52435
BT-31
CSE-AIML

Lab 18 – API Integration: Connecting to External Services with Error Handling
Lab Objectives:
•Learn how to integrate Python programs with external REST APIs.
•Understand API request/response handling using requests or similar libraries.
•Implement proper error handling for failed API calls (timeouts,invalid responses, rate limits).
•Practice extracting and displaying meaningful information from API responses.

Task 1 – Weather API Integration
Connect to a Weather API to fetch current weather details for a given city.
Instructions:
•Prompt AI to generate Python code using the requests library.
•Handle errors: invalid city name, API key errors, and network failures.
•Display temperature, humidity, and weather description in a readable format.
Expected Output:
•A working Python script that fetches and displays weather data with proper error handling.

Prompt:-
Write a Python program that integrates with a Weather API (such as OpenWeatherMap) using the requests library to fetch current weather details for a given city.

```python
import requests

def get_coordinates(city_name):
    """Get latitude and longitude from city name using geocoding API"""
    url = "https://geocoding-api.open-meteo.com/v1/search"
    params = {"name": city_name, "count": 1, "language": "en", "format": "json"}
    response = requests.get(url, params=params)
    data = response.json()

    if data.get("results"):
        result = data["results"][0]
        return result["latitude"], result["longitude"]
    return None, None

def get_weather(latitude, longitude):
    """Get weather data from Open-Meteo API"""
    url = "https://api.open-meteo.com/v1/forecast"
    params = {
        "latitude": latitude,
        "longitude": longitude,
        "current": "temperature_2m,relative_humidity_2m,weather_code",
        "temperature_unit": "fahrenheit"
    }
    response = requests.get(url, params=params)
    return response.json()
```

```python
def fetch_weather(city_name):
    """Fetch weather for a city"""
    lat, lon = get_coordinates(city_name)

    if lat is None:
        print(f"City '{city_name}' not found")
        return

    weather_data = get_weather(lat, lon)
    current = weather_data["current"]

    print(f"Weather in {city_name}")
    print(f"Temperature: {current['temperature_2m']}°F")
    print(f"Humidity: {current['relative_humidity_2m']}%")
    print(f"Weather Code: {current['weather_code']}")

if __name__ == "__main__":
    fetch_weather("Seoul")
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/Ap
Desktop/AIAC/demo113.py
Weather in Seoul
Temperature: 38.2°F
Humidity: 97%
Weather Code: 1
```

Task 2 – Currency Exchange Rates

Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.

Instructions:

•Use AI to generate API request code.

•Validate user input for amount and currency codes.

•Implement error handling for invalid responses and API downtime.

•Show conversion results clearly in tabular format.

Expected Output:

•A script that converts INR to multiple currencies with graceful error handling.

Prompt:-

Write a Python program using requests to convert INR to USD, EUR, and GBP using https://api.exchangerat

Take INR amount as input, validate it, and display results in a table format.

Add error handling for invalid input, API errors, and network issues.

C: > Users > ankam > .vscode > extensions > ms-python.vscode-pylance-2026.1.1 > dist > typeshed-fallback > stubs > tabulate > tabulate >

```python
from collections.abc import Callable, Container, Iterable, Mapping, Sequence
from typing import Any, Final, NamedTuple
from typing_extensions import Self, TypeAlias

from .version import __version__ as __version__

__all__ = ["tabulate", "tabulate_formats", "simple_separated_format"]

# These constants are meant to be configurable
# https://github.com/astanin/python-tabulate#text-formatting
PRESERVE_WHITESPACE: bool
MIN_PADDING: int
# https://github.com/astanin/python-tabulate#wide-fullwidth-cjk-symbols
WIDE_CHARS_MODE: bool
SEPARATING_LINE: str

class Line(NamedTuple):
    begin: str
    hline: str
    sep: str
    end: str

class DataRow(NamedTuple):
    begin: str
    sep: str
    end: str

_TableFormatLine: TypeAlias = None | Line | Callable[[list[int], list[str]], str]
_TableFormatRow: TypeAlias = None | DataRow | Callable[[list[Any], list[int], list[str]], str]
```

```python
class TableFormat(NamedTuple):
    lineabove: _TableFormatLine
    linebelowheader: _TableFormatLine
    linebetweenrows: _TableFormatLine
    linebelow: _TableFormatLine
    headerrow: _TableFormatRow
    datarow: _TableFormatRow
    padding: int
    with_header_hide: Container[str] | None

LATEX_ESCAPE_RULES: Final[dict[str, str]]
tabulate_formats: list[str]
multiline_formats: dict[str, str]

def simple_separated_format(separator: str) -> TableFormat: ...
def tabulate(
    # The key is converted using str().
    tabular_data: Mapping[Any, Iterable[Any]] | Iterable[Iterable[Any]],
    headers: str | dict[str, str] | Sequence[str] = (),
    tablefmt: str | TableFormat = "simple",
    floatfmt: str | Iterable[str] = "g",
    intfmt: str | Iterable[str] = "",
    numalign: str | None = "default",
    stralign: str | None = "default",
    missingval: str | Iterable[str] = "",
    showindex: str | bool | Iterable[Any] = "default",
    disable_numparse: bool | Iterable[int] = False,
    colalign: Iterable[str | None] | None = None,
    maxcolwidths: int | Iterable[int | None] | None = None,
    rowalign: str | Iterable[str] | None = None,
    maxheadercolwidths: int | Iterable[int] | None = None,
) -> str: ...

class JupyterHTMLStr(str):
    @property
    def str(self) -> Self: ...
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>python demo118.py
INR to USD, EUR, GBP Converter
----------------------------------------
Enter amount in INR: 100

₹100.00 INR converts to:

+------------+----------+
| Currency   |   Amount |
+============+==========+
| USD        |     1.06 |
+------------+----------+
| EUR        |     0.92 |
+------------+----------+
| GBP        |     0.8  |
+------------+----------+
```

Task 3 – GitHub Repository Info Fetcher

Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).

Instructions:

•Prompt AI to write a Python function to send GET requests to GitHub API.

•Handle rate limit errors, 404 (repo not found), and invalid input.

•Display repository name, description, stars, forks, and open issues.

Expected Output:

•A Python program that retrieves and displays GitHub repository information with robust error handling.

Prompt:-

Write a Python program using requests to fetch GitHub repository details (name, description, stars, forks, i

Take repository name as input (owner/repo), validate it, and display results clearly.

Add error handling for invalid input, 404 errors, and API rate limits.

demo119.py > fetch_repo_details

```python
import requests
import sys

def validate_repo_input(repo_input):
    """Validate repository input format."""
    if not repo_input or '/' not in repo_input:
        raise ValueError("Invalid format. Use 'owner/repo'")
    parts = repo_input.split('/')
    if len(parts) != 2 or not all(parts):
        raise ValueError("Invalid format. Use 'owner/repo'")
    return parts

def fetch_repo_details(owner, repo):
    """Fetch GitHub repository details using API."""
    url = f"https://api.github.com/repos/{owner}/{repo}"

    try:
        response = requests.get(url, timeout=5)

        if response.status_code == 404:
            raise Exception("Repository not found (404)")
        elif response.status_code == 403:
            raise Exception("API rate limit exceeded. Try again later.")
        elif response.status_code != 200:
            raise Exception(f"API error: {response.status_code}")
```

```python
        data = response.json()
        return {
            'name': data.get('name'),
            'description': data.get('description'),
            'stars': data.get('stargazers_count'),
            'forks': data.get('forks_count'),
            'issues': data.get('open_issues_count')
        }

    except requests.exceptions.Timeout:
        raise Exception("Request timed out. Try again.")
    except requests.exceptions.RequestException as e:
        raise Exception(f"Network error: {str(e)}")

def display_results(repo_data):
    """Display repository details in a formatted way."""
    print("\n" + "="*50)
    print(f"Repository: {repo_data['name']}")
    print("="*50)
    print(f"Description: {repo_data['description'] or 'N/A'}")
    print(f"Stars:       {repo_data['stars']}")
    print(f"Forks:       {repo_data['forks']}")
    print(f"Open Issues: {repo_data['issues']}")
    print("="*50 + "\n")
```

```python

def display_results(repo_data):
    """Display repository details in a formatted way."""
    print("\n" + "="*50)
    print(f"Repository: {repo_data['name']}")
    print("="*50)
    print(f"Description: {repo_data['description'] or 'N/A'}")
    print(f"Stars:       {repo_data['stars']}")
    print(f"Forks:       {repo_data['forks']}")
    print(f"Open Issues: {repo_data['issues']}")
    print("="*50 + "\n")

def main():
    try:
        repo_input = input("Enter repository (owner/repo): ").strip()
        owner, repo = validate_repo_input(repo_input)

        print("Fetching repository details...")
        repo_data = fetch_repo_details(owner, repo)
        display_results(repo_data)

    except ValueError as e:
        print(f"Error: {str(e)}")
        sys.exit(1)
    except Exception as e:
        print(f"Error: {str(e)}")
        sys.exit(1)

if __name__ == "__main__":
    main()
```

OUTPUT:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Program
Enter repository (owner/repo): 2303A52435/2435-AIAC
Fetching repository details...

Enter repository (owner/repo): 2303A52435/2435-AIAC
Fetching repository details...

Fetching repository details...


==================================================
Repository: 2435-AIAC
==================================================
Description: N/A
Stars:       0
Forks:       0
Open Issues: 0
==================================================


C:\Users\ankam\OneDrive\Desktop\AIAC>
```

Task 4 – Real-Time Application: News Headlines Aggregator
Scenario: Build a News Aggregator using a free News API.
Requirements:
•Fetch top 5 headlines for a given category (sports, technology, health).
•Handle errors like invalid category, missing API key, and request failures.
•Display headlines in a user-friendly numbered list format.
•Include a retry mechanism if the first request fails.
Expected Output:
•A working Python script that retrieves and displays news headlines with proper error handling

Prompt:-
Write a Python program using requests to fetch top 5 news headlines for a given category (sports, technolo
Take category as input, validate it, and display headlines in a numbered list.
Add error handling for invalid category, missing API key, request failures, and include a retry mechanism.

demo120.py > fetch_news

```python
import requests
import time

def fetch_news(category):
    api_key = "7a835486bde54f8a8024c694f61493eb"  # 🔴 Replace with your NewsAPI key
    url = f"https://newsapi.org/v2/top-headlines?country=us&category={category}&apiKey={api_key}
    retries = 2  # retry mechanism

    for attempt in range(retries + 1):
        try:
            response = requests.get(url, timeout=5)
            response.raise_for_status()
            data = response.json()

            # Check API response
            if data.get("status") != "ok":
                print("Error:", data.get("message", "Unknown error"))
                return

            articles = data.get("articles", [])

            if not articles:
                print("No news found.")
                return

            print(f"\nTop 5 {category.capitalize()} Headlines:")
            print("-" * 40)

            for i, article in enumerate(articles[:5], 1):
                print(f"{i}. {article.get('title', 'No Title')}")
```

```python
def fetch_news(category):
            for i, article in enumerate(articles[:5], 1):
                print(f"{i}. {article.get('title', 'No Title')}")

            print("-" * 40)
            return

        except requests.exceptions.RequestException as e:
            print(f"Attempt {attempt+1} failed:", e)
            if attempt < retries:
                print("Retrying...\n")
                time.sleep(2)
            else:
                print("Failed to fetch news. Please try again later.")

# ---- MAIN PROGRAM ----
category = input("Enter category (sports, technology, health): ").strip().lower()

# Validate category
valid_categories = ["sports", "technology", "health"]

if category not in valid_categories:
    print("Error: Invalid category! Choose from sports, technology, health.")
else:
    fetch_news(category)
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python/Python312/python.exe c:/User
neDrive/Desktop/AIAC/demo120.py
Enter category (sports, technology, health): technology

Top 5 Technology Headlines:
No news found.

C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python/Python312/python.exe c:/User
neDrive/Desktop/AIAC/demo120.py
Enter category (sports, technology, health): technology

Top 5 Technology Headlines:
Enter category (sports, technology, health): technology

Top 5 Technology Headlines:
Top 5 Technology Headlines:
----------------------------------------
1. I Tried Some Games With Nvidia's DLSS 5. It's Controversial, Yes, But Also an Eyeful - PCMag
2. Epic Games lays off over 1,000 employees following "downturn in Fortnite engagement" - GamesIndustry.biz
3. Pokémon Champions Releases on Nintendo Switch and Nintendo Switch 2 on April 8, 2026 - Pokemon.com
4. Exclusive: Samsung Galaxy Z Fold 8 Official CAD Renders & Rumors - Android Headlines
5. Xbox Game Pass gets a new cozy game with unsettling vibes this week - TrueAchievements
----------------------------------------
```